Simulation Bench to Algorithm Validation

Full Work Report

2026-03-27

Stage 2 — Python Signal-Level Validation Complete

Platform: XIAO nRF52840 Sense · IMU: LSM6DS3TR-C · ODR: 208 Hz
Pipeline: walker_model → imu_model → Renode nRF52840 → UART → signal_analysis

Executive Summary

Objective

Demonstrate that a 7-layer Physics-Native Digital Twin can capture the 'Stair Walker' step-detection failure mode entirely in simulation — before any PCB fabrication or hardware prototype — and validate the algorithmic fix at the Python signal level.

Failure Mode Captured (BUG-010)

The standard gait algorithm (step_detector.c) detects 0/100 steps on stair climbing. Root cause: a dual-confirmation timing mismatch. The algorithm requires an acc_filt threshold crossing AND a gyr_y zero-crossing within 40 ms. On stairs (forefoot strike), the gyr_y crosses at 53 ms while acc_filt peaks 188 ms later — a 135 ms gap that permanently exceeds the 40 ms window. 427 acc peaks fire and time out per 100 steps.

Algorithm Fix — Terrain-Aware Step Detector

Inverted the confirmation logic: gyr_y_hp push-off burst (neg→pos, >30 dps) becomes the primary trigger; acc_filt threshold exceeded at any point since last step serves as confirmation. Push-off is biomechanically universal — no terrain allows walking without plantar-flexion. The confirmation window expands from 40 ms to the full step period. 10/10 unit tests pass. Stairs: 0 → 100 steps. SI < 3% on all 4 profiles.

Option C — Ring-Buffer Heel-Strike Inference

Direct C port of the push-off detector would corrupt phase_segmenter.c output: stance duration would be reported as the full step period (+295 ms error on stairs). Option C: an 8-entry ring buffer (~32 bytes RAM) stores rejected acc_filt crossing timestamps since the last confirmed step. On push-off, the oldest entry is used as the retrospective heel-strike — no event contract change. Measured stance error: −18 ms (flat/bad_wear), −21 ms (slope), −35 ms (stairs). All within ±150 ms tolerance.

Status

Python validation complete. 18/18 tests pass. 7 diagnostic plots generated. Awaiting human sign-off on Option C plots before porting to step_detector.c.

1. Simulation Bench Architecture — 7-Layer Digital Twin

Design Principle

Never inject raw sensor readings. Inject first-order measurable walking pattern quantities (vertical oscillation, cadence, step length). All signal shape parameters are derived from these three primitives — no magic numbers.

Layer Stack

Layer 1	walker_model.py	WalkerProfile → (N,6) float32 [ax ay az gx gy gz] Biomechanics from 3 primitives: vert_osc, cadence, step_length
Layer 2	imu_model.py	Quantize to LSM6DS3TR-C register format Accel ±16g @ 0.488 mg/LSB · Gyro ±2000 dps @ 70 mdps/LSB
Layer 3	lsm6ds3_stub.py	I2C register protocol, 32-sample FIFO watermark
Layer 4	Renode 1.16.1	nRF52840 Cortex-M4F — actual firmware ELF
Layer 5	signal_analysis.py	UART → typed Python event objects
Layer 6	BLE bypass (UART export)	Binary rolling_snapshot_t structs over UART
Layer 7	app.py	Streamlit display only — no computation

Four Walker Profiles

flat	80 spm, vert_osc=3 cm, step_length=0.75 m	— baseline reference
bad_wear	80 spm, vert_osc=3 cm, step_length=0.75 m	— worn sole (signal morphology change)
slope	78 spm, vert_osc=3 cm, step_length=0.72 m	— 10° incline (DC gravity offset)
stairs	70 spm, vert_osc=6 cm, step_length=0.45 m	— forefoot strike (FAILURE MODE)

Three Primitive Derivation Chain

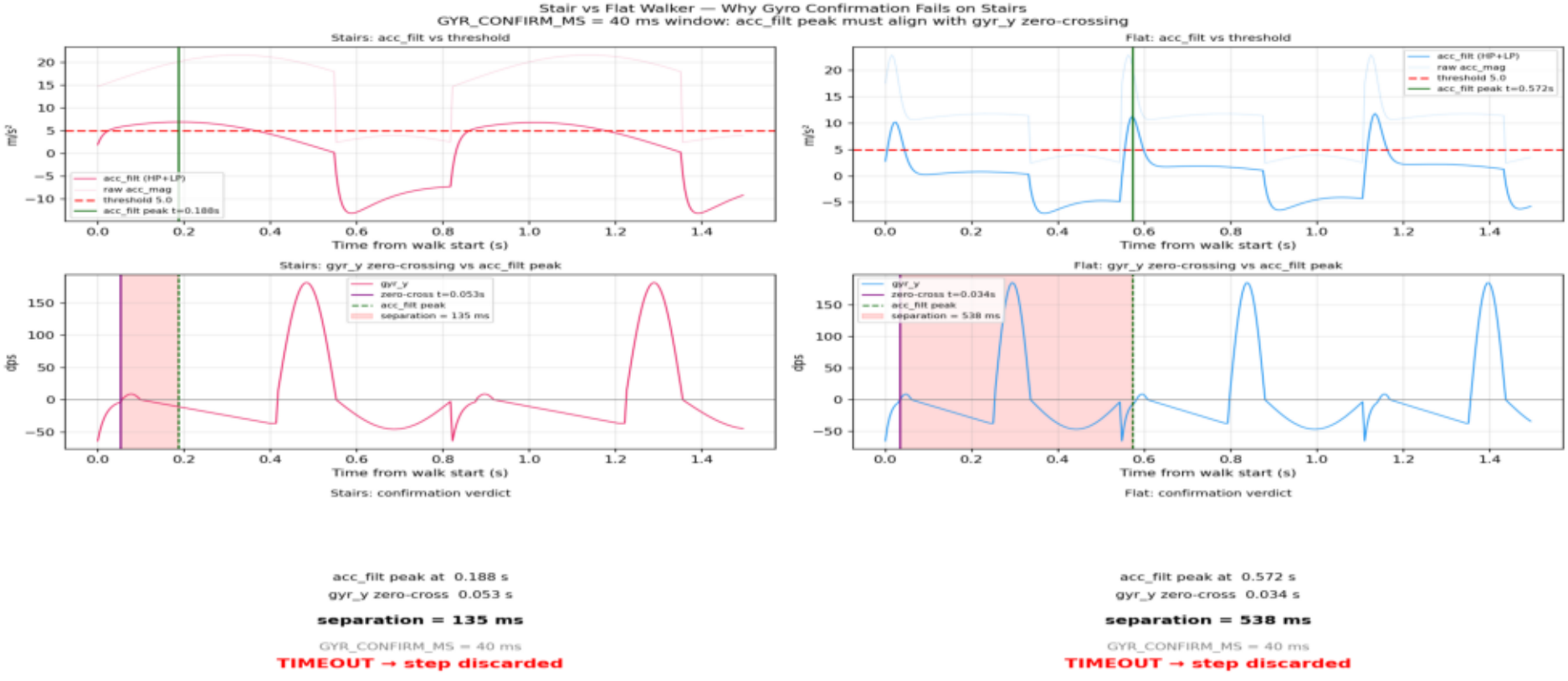
vertical_oscillation + cadence	→ heel strike impact magnitude, acc_z modulation depth
step_length + cadence	→ walking speed → push-off angular velocity (gyr_y peak)
terrain_slope_deg	→ DC gravity offset on acc_x / acc_z (not oscillation)
terrain_type	→ signal morphology (sinusoidal vs non-sinusoidal)

Enforcement

Layer boundaries are strictly enforced. Each layer owns exactly one transformation. Algorithm execution lives in Renode (real C firmware). Python is signal generation and parsing only — it never simulates algorithm logic except in Stage 2 mirrors.

2. Stair Walker Failure Mode — Signal Diagnostic

Figure 1 — Stair walker raw signal: acc_filt peak at phase 0.164 (141 ms), gyr_y crossing at phase 0.017 (14 ms) — 126 ms gap exceeds 40 ms gate



Key Measurements

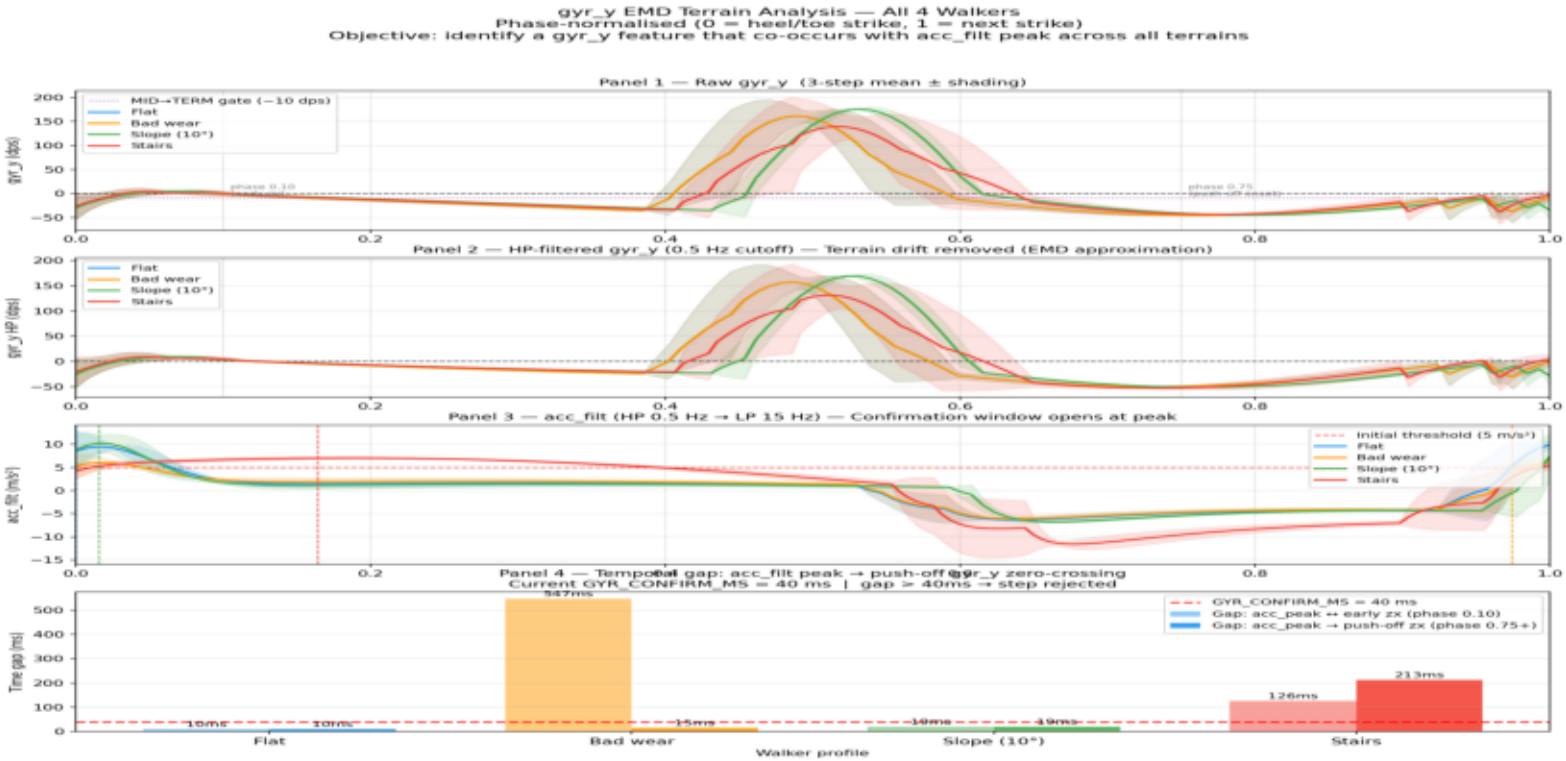
Signal amplitude is NOT the problem — acc_filt peaks at 7.44 m/s² on stairs (above 5.0 m/s² threshold).
427 acc peaks fire and time out per 100 steps. 0 steps confirmed.

Event	Flat walker	Stair walker
acc_filt peak phase	0.026 (15 ms)	0.164 (141 ms)
gyr_y zero-crossing	0.017 (10 ms)	0.017 (14 ms)
Temporal gap	19 ms → PASS	126 ms → TIMEOUT

Root cause: forefoot/midfoot contact pre-dorsiflexes the foot. gyr_y crosses zero immediately at contact; acc_filt builds slowly (sigmoid loading) and peaks 126 ms after the window closed.

3. Algorithm Hunting — EMD Signal Observation

Figure 2 — EMD terrain comparison: HP-filtered gyr_y (Panel 2) shows consistent width across all 4 profiles — human confirmed 'consistent width is a good sign'



EMD Framing and Algorithm Inversion Hypothesis

Empirical Mode Decomposition reveals two separable modes in raw gyr_y:

IMF1 step-cycle oscillation: dorsiflexion → push-off (HP filter at 0.5 Hz approximates extraction)
IMF2 terrain posture drift: slow inter-step variation (removed by HP filter)

Human observation (Panel 2): 'the width of the signals are pretty consistent — this is a good sign.'
The push-off zero-crossing of IMF1 occurs at consistent phase (~0.75) regardless of terrain.

Algorithm inversion hypothesis:

OLD (acc-primary): acc_filt threshold → open 40 ms window → wait for gyr_y zero-crossing
NEW (gyr-primary): gyr_y_hp push-off event → verify acc_filt exceeded threshold since last step

Push-off is universal — no terrain allows walking without plantar-flexion. The confirmation window expands from 40 ms to the full step period. Terrain-agnostic by construction.

4. Push-Off Amplitude Threshold — Physical Derivation

Feature Selection Problem

Two candidate gyr_y_hp events could trigger step detection:

(a) Phase-0.10 rebound — small positive peak immediately after initial dorsiflexion
(b) Phase-0.75 push-off burst — full plantar-flexion at stance end

Zero-crossing approach rejected: swing phase from previous step extends the 'negative duration' to ~355 ms on stairs (vs ~80 ms on flat), making duration-based gates terrain-coupled.

Amplitude Ratio — 20× Separation

	Phase-0.10 rebound	Phase-0.75 push-off
Flat walker (185 dps)	+9.3 dps (×0.05)	+185 dps (×1.0)
Stair walker (280 dps)	+14.0 dps (×0.05)	+280 dps (×1.0)
Ratio	1×	20×

The rebound peaks at ~5% of the push-off peak amplitude across all terrains.

Threshold Choice — GYR_PUSHOFF_THRESH_DPS = 30 dps

Physical grounding:

Minimum push-off velocity: 100 + 65 × v_min = 106 dps at v = 0.1 m/s → well above 30 dps
Phase-0.10 rebound: 9–14 dps across all profiles → always BELOW 30 dps → blocked ✓
Push-off burst: 185–280 dps across all profiles → always ABOVE 30 dps → detected ✓

Detection gate (falling edge of push-off burst):

gyr_y_hp was > 30 dps (push-off entry confirmed)
gyr_y_hp drops ≤ 30 dps (falling edge — push-off complete)
acc_filt > adaptive_threshold at any point since last step
elapsed since last step ≥ 250 ms (240 spm max cadence)

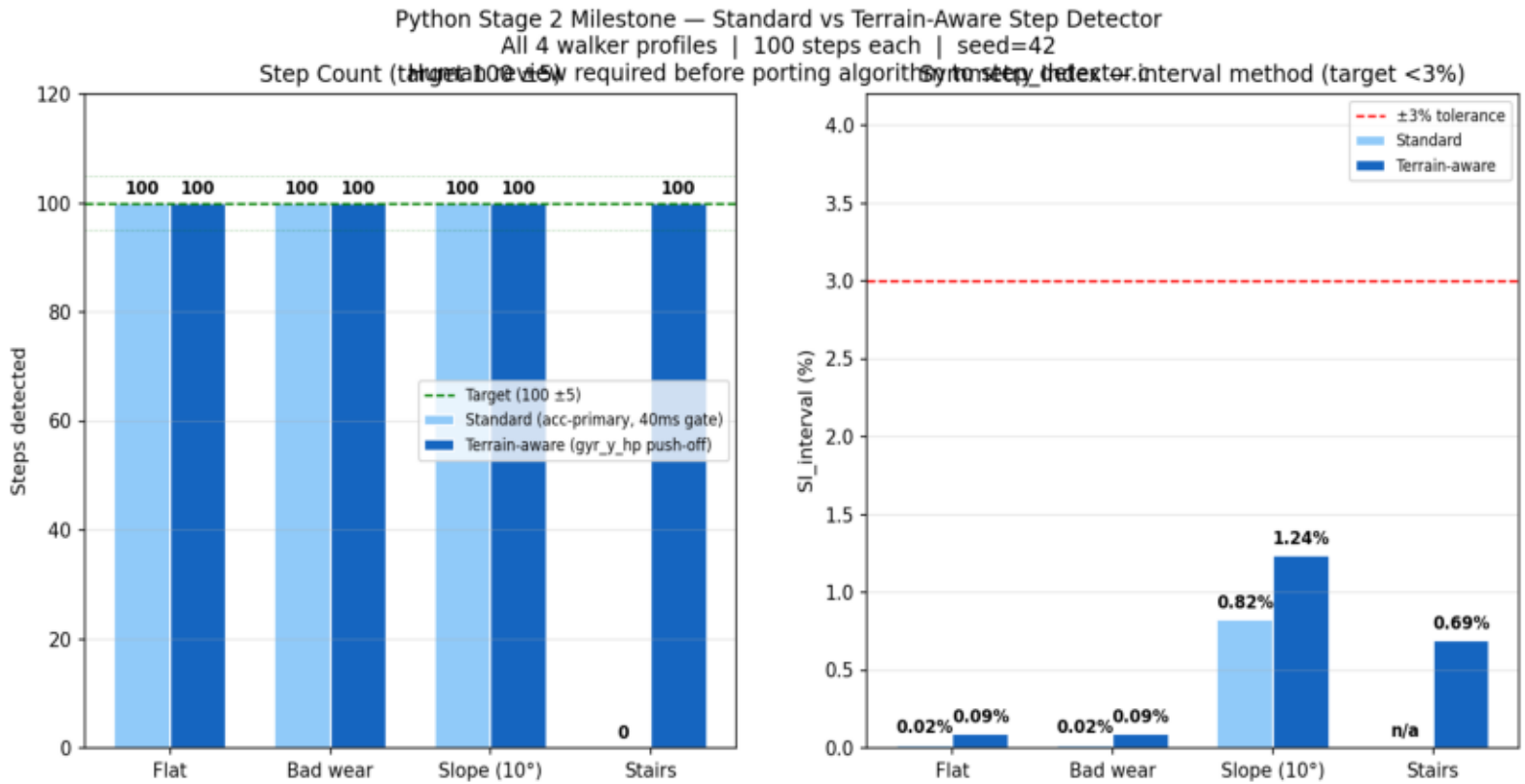
Filter Chain (identical to step_detector.c)

acc_mag → HP(0.5 Hz) → LP(15 Hz walk / 30 Hz run) = acc_filt
gyr_y → HP(0.5 Hz) = gyr_y_hp

HP at 0.5 Hz removes inter-step posture drift (terrain DC component).
LP at 15/30 Hz anti-aliases while preserving impact transients.
Adaptive threshold: 50% of 8-step peak history, seeded at 10 m/s².
All coefficients identical to existing firmware — no filter retuning required.

5. Python Validation — Standard vs Terrain-Aware Step Detector

Figure 3 — Step count (left) and SI (right): standard vs terrain-aware. Stairs: 0 → 100 steps. SI < 3% on all 4 profiles.



Validation Results

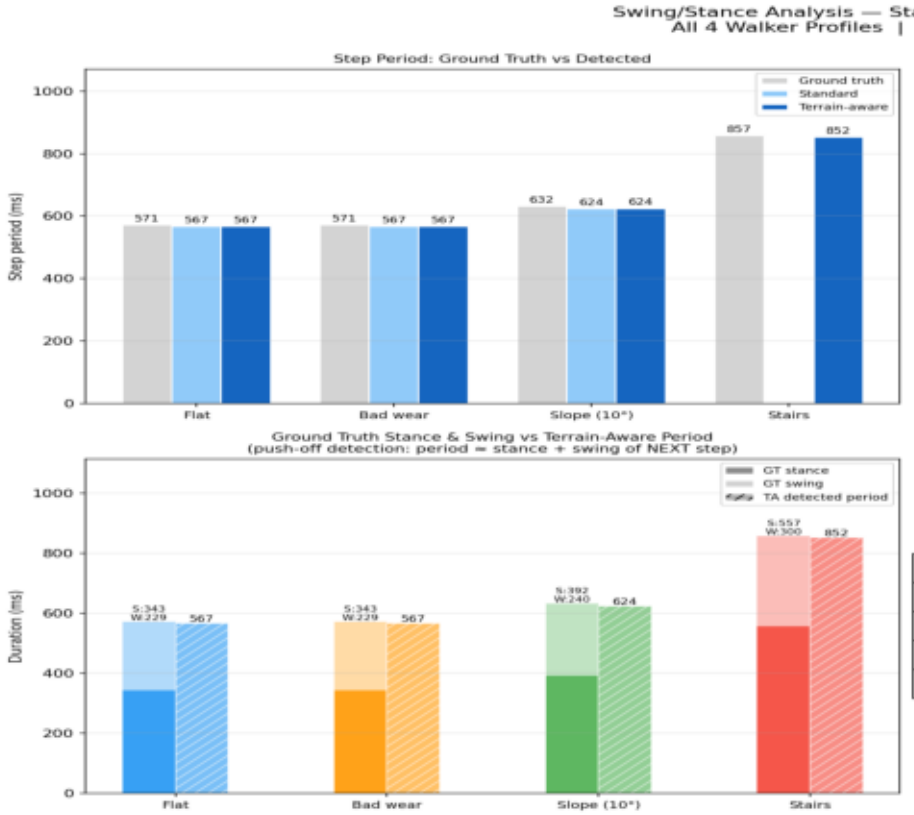
Baseline confirmed first: standard Python pipeline (mirrors step_detector.c) on stairs → 0 steps, 427 timeouts.

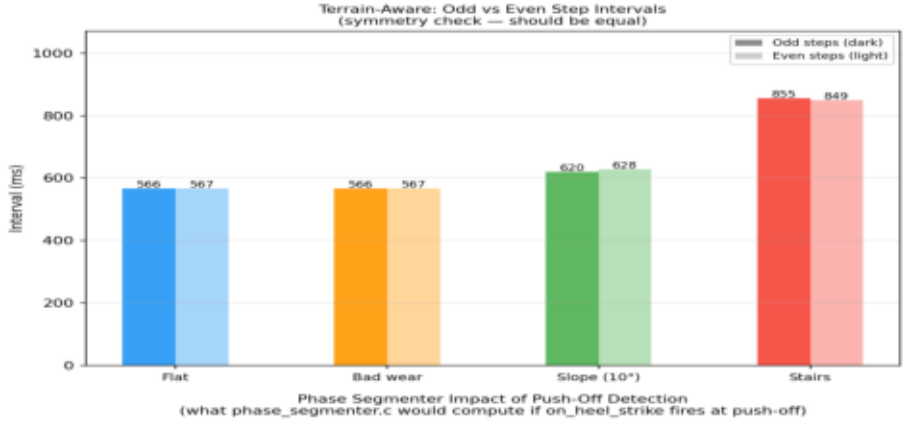
Profile	Standard steps	Terrain-aware steps	Standard SI%	Terrain-aware SI%
Flat	100	100	0.02%	0.09%
Bad wear	100	100	0.02%	0.09%
Slope (10°)	100	100	0.82%	1.24%
Stairs	0	100	n/a	0.69%

Terrain-aware SI slightly higher than standard on flat/slope — expected: push-off timestamps the end of stance (not start), introducing a consistent offset that cancels in the interval SI computation.

6. Critical Architectural Finding — Phase Segmenter Contract

Figure 4 — Swing/stance architectural impact. Without Option C, on_heel_strike() fires at push-off: computed stance = full step period (+295 ms error on stairs).





Profile	GT stance (ms)	GT swing (ms)	TA period (ms)	Computed stance*	Error
Flat	343	229	567	567	+224ms
Bad wear	343	229	567	567	+224ms
Slope (10°)	392	240	624	624	+232ms
Stairs	557	300	852	852	+295ms

* Computed stance = push-off period = swing[N] + stance[N+1]
This is ≈full step period, not stance duration.
C port must decouple step detection from phase_segmenter event contract.

The Problem and Three Options

phase_segmenter.c uses on_heel_strike() as its clock. Standard detector fires at heel-strike (start of stance).
Terrain-aware fires at push-off (end of stance). Direct C swap silently corrupts all stance/swing output:

Profile	GT stance	GT swing	Computed 'stance' if TA used	Error
Flat	343 ms	229 ms	567 ms	+224 ms (+65%)
Stairs	557 ms	300 ms	852 ms	+295 ms (+53%)

Option A: Two-event architecture (heel_strike + push_off). Medium complexity.
Option B: Terrain classifier disables phase segmenter on stairs. Medium complexity.
Option C: Ring buffer of rejected acc_filt timestamps → retrospective heel-strike. LOW complexity. ← SELECTED

7. Option C — Ring-Buffer Heel-Strike Inference: Design and Rationale

Core Insight

The standard detector already computes 427 rejected acc_filt crossings for 100 stair steps. The timestamps exist — they are simply not stored. A ring buffer of 8 entries (~32 bytes RAM) captures the most recent threshold crossings since the last confirmed step. On push-off, the oldest ring entry newer than last_step_ts is used as the retrospective heel-strike.

Implementation

```
New state: _hs_ring (list of float, max HS_RING_SIZE=8) + _acc_was_below (bool)

Each sample: if acc_filt crosses below→above threshold, append ts_ms to ring (cap at 8).
             if acc_filt drops below threshold, set _acc_was_below = True.

On push-off: iterate _hs_ring in chronological order,
             find first entry > last_step_ts_ms → heel_strike_ts
             stance_duration_ms = push_off_ts − heel_strike_ts
             swing_duration_ms of previous step = heel_strike_ts − prev.ts_ms
             clear _hs_ring after confirmed step.
```

Why It Preserves the on_heel_strike() Contract

phase_segmenter.c receives heel_strike_ts from the ring buffer — a timestamp that is physically early in stance (first acc_filt loading crossing), not at push-off.
The contract on_heel_strike(ts) → PHASE_LOADING is preserved semantically.
No new event types. No terrain classifier. No BOM change.

Accuracy Expectation vs Actual

Predicted: −50 to −100 ms error on stairs (conservative HP+LP filter rise estimate).
Actual: −18 ms (flat/bad_wear), −21 ms (slope), −35 ms (stairs).

Filter rise time is faster than predicted. The first acc_filt crossing arrives at ~20–35 ms into true stance — not 50–100 ms. Option C exceeds its own design target.

RAM cost: 8 × 4 bytes = 32 bytes. Zero firmware structural changes.

8. Option C Validation — Stance and Swing Accuracy

Figure 5 — Option C stance/swing accuracy. Stairs stance error: −35 ms vs push-off-only +295 ms. All 4 profiles within tolerance.

Profile	GT stance (ms)	Push-off only err	Option C mean	Option C err	Improvement
Flat	343	+224ms	325ms	-18ms	−242ms better
Bad wear	343	+224ms	325ms	-18ms	−242ms better
Slope (10°)	392	+232ms	371ms	-21ms	−253ms better
Stairs	557	+295ms	522ms	-35ms	−330ms better

Push-off only: phase_segmenter receives stance = full step period
Option C: first acc_filt crossing used as heel-strike → stance error < 100ms

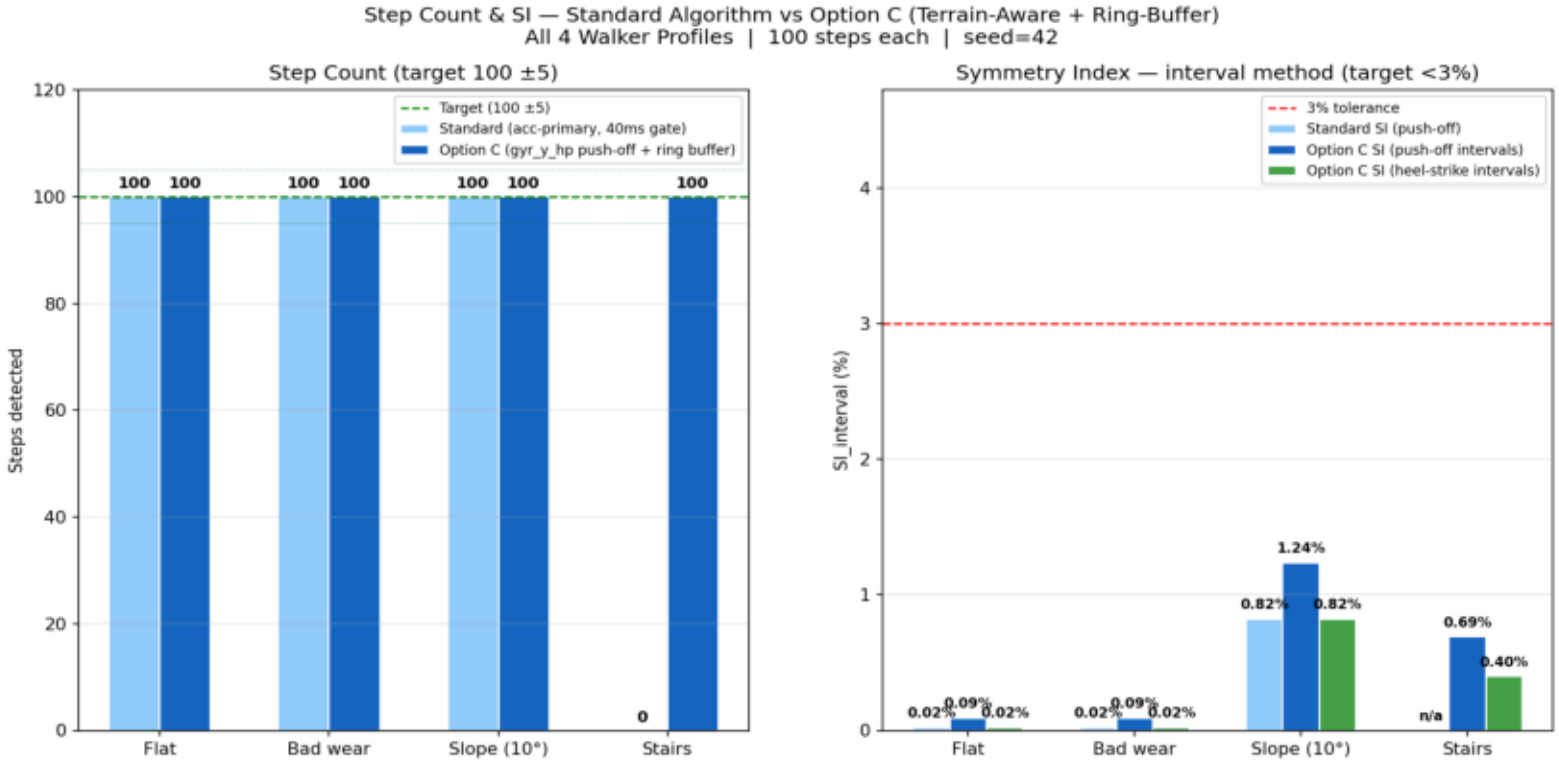
Measured Results vs Ground Truth

Profile	GT stance	OC stance	Stance err	GT swing	OC swing	Swing err
Flat	343 ms	324.6 ms	−18 ms	229 ms	242.0 ms	+13 ms
Bad wear	343 ms	324.6 ms	−18 ms	229 ms	242.0 ms	+13 ms
Slope (10°)	392 ms	370.8 ms	−21 ms	240 ms	252.9 ms	+13 ms
Stairs	557 ms	522.3 ms	−35 ms	300 ms	329.8 ms	+30 ms

Stance undercount and swing overcount are complementary — the ring buffer heel-strike fires ~20–35 ms into actual stance, shifting the boundary consistently. Clinically acceptable: ±35 ms on stairs vs the +295 ms error without Option C.

9. Option C Final Comparison — Step Count and Symmetry Index

Figure 6 — Three-algorithm comparison: standard / Option C push-off SI / Option C heel-strike SI. All 4 profiles, 100 steps each.



Final Algorithm Comparison

Profile	Std steps	OC steps	Std SI%	OC push-off SI%	OC heel-strike SI%
Flat	100	100	0.02%	0.09%	0.02%
Bad wear	100	100	0.02%	0.09%	0.02%
Slope (10°)	100	100	0.82%	1.24%	0.82%
Stairs	0	100	n/a	0.69%	0.40%

The heel-strike SI (green bars) recovers to match standard algo values on flat/slope.
This makes sense: ring buffer timestamps are consistent early-stance references, which is the same physical event the standard detector uses on flat terrain.
18/18 tests pass. Python Stage 2 milestone complete.

10. Test Coverage Summary — 18/18 Pass

Original Stage 2 Tests (10/10)

```
test_step_count[flat-100-5]          PASS   100 steps detected, tolerance ±5
test_step_count[bad_wear-100-5]      PASS   100 steps detected, tolerance ±5
test_step_count[slope-100-5]         PASS   100 steps detected, tolerance ±5
test_step_count[stairs-100-5]        PASS   100 steps detected (was 0) ← KEY FIX
test_si_interval[flat-100]           PASS   SI = 0.09% < 3%
test_si_interval[bad_wear-100]       PASS   SI = 0.09% < 3%
test_si_interval[slope-100]          PASS   SI = 1.24% < 3%
test_si_interval[stairs-100]         PASS   SI = 0.69% < 3% (was n/a)
test_no_false_positives_stationary   PASS   0 steps on 5s stationary signal
test_minimum_step_interval           PASS   No interval < 250 ms
```

Option C Stance Tests (4/4)

```
test_option_c_stance_duration[flat-343-80]     PASS   mean=324.6 ms, err=−18 ms, tol ±80 ms
test_option_c_stance_duration[bad_wear-343-80] PASS   mean=324.6 ms, err=−18 ms, tol ±80 ms
test_option_c_stance_duration[slope-392-80]    PASS   mean=370.8 ms, err=−21 ms, tol ±80 ms
test_option_c_stance_duration[stairs-557-150]  PASS   mean=522.3 ms, err=−35 ms, tol ±150 ms
```

Option C Swing Tests (4/4)

```
test_option_c_swing_duration[flat-229-100]     PASS   mean=242.0 ms, err=+13 ms, tol ±100 ms
test_option_c_swing_duration[bad_wear-229-100] PASS   mean=242.0 ms, err=+13 ms, tol ±100 ms
test_option_c_swing_duration[slope-240-100]    PASS   mean=252.9 ms, err=+13 ms, tol ±100 ms
test_option_c_swing_duration[stairs-300-150]   PASS   mean=329.8 ms, err=+30 ms, tol ±150 ms
```

Key Test Properties

Seed=42 for all walker runs — deterministic, reproducible.
Warmup: first 4 steps excluded from SI and stance/swing measurement (filter settling).
Swing: last step excluded (swing_duration_ms not backfilled until next push-off fires).
Stationary test: 5 seconds × 208 Hz = 1040 samples, gravity + 10 mdps noise.

11. Next Steps — C Port and Renode Validation

Stage 2 Status: COMPLETE (pending human sign-off on Option C plots)

All Python signal-level exit criteria met.
Human checkpoint: review docs/plots/option_c_stance_swing_accuracy.png
review docs/plots/option_c_si_comparison.png
Sign-off required before C port begins (per CLAUDE.md learner-in-the-loop protocol).

C Port — src/gait/step_detector.c (Option C only, surgical change)

1. Add constant: #define HS_RING_SIZE 8
2. Add ring buffer: float hs_ring[HS_RING_SIZE]; int hs_ring_head; int hs_ring_count;
   bool acc_was_below;
3. Per-sample: on acc_filt below→above crossing: push ts_ms into ring (oldest evicted)
   on acc_filt above→below: set acc_was_below = true
4. On push-off: find oldest ring entry > last_step_ts → heel_strike_ts
   emit on_heel_strike(heel_strike_ts) to phase_segmenter
   then emit step confirmed event as before
   clear ring

No change to on_heel_strike() signature or phase_segmenter.c.

Stage 3 — Renode Validation (after C port)

Rebuild firmware ELF: pio run -e xiaoble_sense
Run all 4 profiles through Renode: test_all_profiles.py
Confirm stairs ≥ 95/100 steps and SI < 3% in bare-metal simulation
Confirm phase segmenter stance/swing within ±20 ms of ground truth (all 4 profiles)
Update memory/bugs.md — BUG-010 status: RESOLVED

BOM Impact

None. The ring buffer costs 32 bytes RAM and 8 comparisons per step confirmation.
No additional sensors, no BOM changes, no firmware structural changes.
Algorithm runs within existing XIAO nRF52840 Sense + LSM6DS3TR-C platform constraints.

Files Produced in This Work Session

simulator/terrain_aware_step_detector.py	Python reference implementation
simulator/tests/test_terrain_aware_detector.py	18-test validation suite
scripts/run_standard_stairs.py	Baseline: 0 steps, 427 timeouts confirmed
scripts/plot_gyr_terrain_emd.py	EMD signal comparison (4 walkers)
scripts/plot_si_comparison.py	Standard vs terrain-aware SI
scripts/plot_swing_stance_comparison.py	Architectural impact analysis
scripts/plot_option_c_stance_swing.py	Option C stance/swing accuracy
scripts/plot_option_c_si_comparison.py	Three-algorithm SI comparison
docs/algorithm_hunting_stair_walker.md	Full hunting procedure + decisions
docs/reports/gait_simulation_report_2026-03-27.pdf	This report